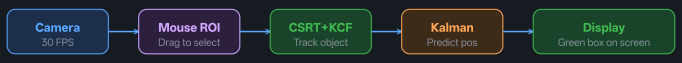
🎯 Object Tracker — Complete Summary with Math

1. System Overview

Your system combines 6 algorithms working together to track any object selected by mouse drag while video plays continuously.



2. ThreadedCamera — Normal Speed Playback

The Problem

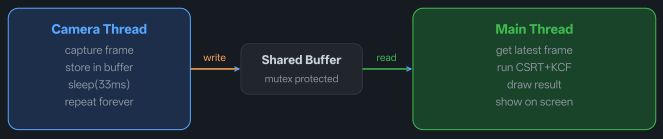
Without threading video plays too fast because there is no delay between frames.

The Math — Frame Delay Calculation

Target delay per frame:

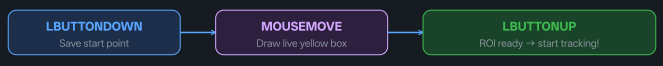
```
Target_MS = 1000 / FPS Example: FPS = 30 Target_MS = 1000 / 30 = 33 milliseconds per frame Sleep_MS = Target_MS - Time_Capture_Took Example: Capture took 5ms Sleep_MS = 33 - 5 = 28ms ← thread sleeps this long
```

Thread Architecture



3. Mouse Drag ROI Selection

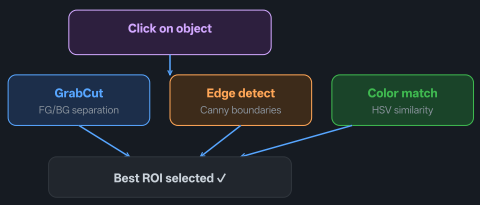
Three Mouse Events



ROI Calculation Math

Start point: (x1, y1) = where mouse was pressed End point: (x2, y2) = where mouse was released ROI.x = min(x1, x2) ← leftmost point ROI.y = min(y1, y2) ← topmost point ROI.width = |x2 - x1| ← absolute difference ROI.height = |y2 - y1| ← absolute difference Example: press at (100,80), release at (250,200) ROI = (100, 80, 150, 120) ← x, y, width, height

4. Auto ROI — 3 Methods



GrabCut Math

GrabCut minimizes energy function: $E = \lambda \cdot R(\alpha) + B(\alpha, z)$ Where: $R(\alpha)$ = Regional term – how likely pixel belongs to FG/BG $B(\alpha)$ = Boundary term – penalizes cuts across strong edges λ =

```
Balance weight between the two terms α
= Label (0=background, 1=foreground)
Result: pixels labeled 1 = your object
✓
```

Color Similarity Math

```
For each pixel P around click point:
HDiff = |P.Hue - Click.Hue| SDiff =
|P.Sat - Click.Sat| VDiff = |P.Val -
Click.Val| If HDiff > 90: HDiff = 180 -
HDiff (hue wraps at 180) Accept pixel
if: HDiff ≤ 15 AND SDiff ≤ 60 AND VDiff
≤ 60 ROI = boundingRect(all accepted
pixels) Width = MaxX - MinX of accepted
pixels Height = MaxY - MinY of accepted
pixels
```

5. CSRT Tracker — Primary Tracker

CSRT stands for Channel and Spatial Reliability Tracking. It is the most accurate tracker in OpenCV contrib.

How CSRT Works — Math

CSRT learns a filter H that maximizes correlation with target: Step 1 – Desired response (Gaussian peak at target center): g(x,y) = exp(-(x² + y²) / (2σ²)) Step 2 – Learn filter H using Ridge Regression: H = (X^T · W · X + λI)^(-1) · X^T · W · g Where: X = Feature matrix (HOG + color channels) W = Spatial reliability weight matrix ← CSRT unique! λ = Regularization parameter g = Desired Gaussian response Step 3 – Find object in next frame: Response = IFFT(FFT(H*) ⊙ FFT(F)) Object location = argmax(Response) The W matrix is CSRT's secret – it reduces weight of background pixels inside the bounding box! ✓

CSRT vs KCF Accuracy

Tracker	VOT Accuracy	FPS	Scale	Occlusion
MOSSE	0.338	669	❌	❌
KCF	0.447	172	❌	❌
MIL	0.412	38	❌	⚠️

CSRT	**0.643**	25	✅	✅

6. Kalman Filter — Prediction During Occlusion

When the object is hidden the Kalman Filter predicts where it should be using physics of motion.

6-State Model

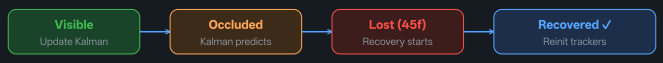
```
State vector: x = [px, py, w, h, vx, vy]^T Where: px, py = position (x, y) w, h = bounding box size vx, vy = velocity in x and y directions ← key!
```

Predict Step — Physics of Motion

```
Transition matrix F: ┌ 1 0 0 0 1 0 ┐ px_new = px + vx │ 0 1 0 0 0 1 │ py_new = py + vy │ 0 0 1 0 0 0 │ w_new = w │ 0 0 0 1 0 0 │ h_new = h │ 0 0 0 0 1 0 │ vx_new = vx (constant vel) └ 0 0 0 0 0 1 ┘ vy_new = vy Prediction: x_pred = F · x_prev Covariance: P_pred = F · P_prev · F^T + Q Q = process noise (1e-4) – how much we trust physics
```

Update Step — Correct With Measurement

When CSRT gives us measurement z = [px, py, w, h]: Innovation: y = z - H · x_pred Gain: K = P_pred · H^T · (H · P_pred · H^T + R)^(-1) Correction: x = x_pred + K · y Covariance: P = (I - K · H) · P_pred R = measurement noise (1e-2) — how noisy CSRT output is K = Kalman gain — balances prediction vs measurement Small R → trust measurement more Large R → trust prediction more



7. Triple Recovery System

IsCorrectObject Math

For each candidate box C: 1. Color check: score = compareHist(ObjHistogram, CandidateHistogram) Accept if score ≥ 0.45 (45% similarity) 2. Size check: WRatio = C.width / OriginalWidth HRatio = C.height / OriginalHeight Accept if 0.5 ≤ WRatio ≤ 2.0 AND 0.5 ≤ HRatio ≤ 2.0 BOTH checks must pass → correct object ✓ Either fails → wrong object, rejected ✗

Template Matching Math

TM_CCOEFF_NORMED formula: $R(x,y) = \Sigma(T(x',y') - \bar{T}) \cdot (F(x+x', y+y') - \bar{F}(x,y))$

$\sqrt{[\Sigma(T(x',y') - \bar{T})^2 \cdot \Sigma(F(x+x', y+y') - \bar{F})^2]}$ Where: T = Template (saved when object selected) F = Current frame being searched $\bar{T}$ = Mean of template $\bar{F}$ = Local mean of frame patch Result: $R(x,y) \in [-1, 1]$ R = 1.0 → perfect match R = 0.0 → no match R = -1.0 →

inverse match Accept if R ≥ 0.55 (55% confidence)

8. 3D Color Histogram — Object Identity

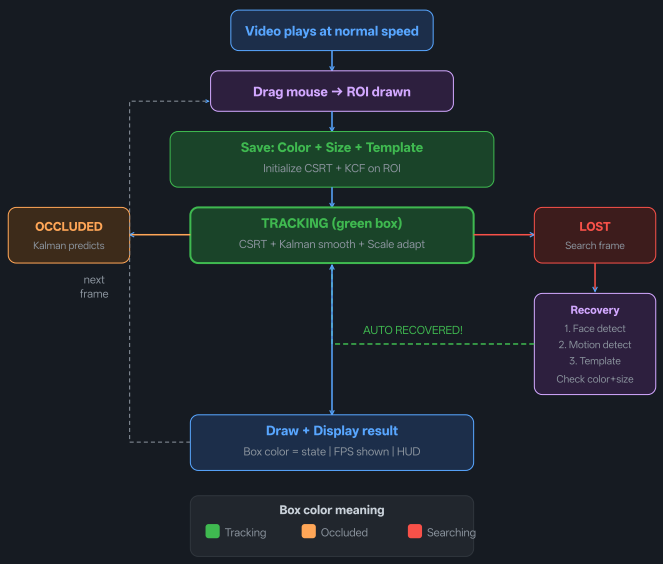
Convert frame to HSV: H ∈ [0, 180] (Hue – color type) S ∈ [0, 256] (Saturation – color purity) V ∈ [0, 256] (Value – brightness) 3D Histogram with 8 bins each dimension: Total bins = 8 × 8 × 8 = 512 bins Each bin counts pixels in that H,S,V range: hist[h][s][v] = count of pixels with Hue in [h·22, (h+1)·22] Sat in [s·32, (s+1)·32] Val in [v·32, (v+1)·32] Compare two histograms using correlation: $d(H1, H2) = \frac{\Sigma(H1_i - \bar{H}1)(H2_i - \bar{H}2)}{\sqrt{[\Sigma(H1_i - \bar{H}1)^2 \cdot \Sigma(H2_i - \bar{H}2)^2]}}$ Result ∈ [-1, 1] 1.0 = identical colors (same object!) 0.0 = no color similarity

9. Scale Adaptation

Handles object getting closer or farther from camera.

```
Try 5 scale factors: S ∈ {0.8, 0.9,
1.0, 1.1, 1.2} For each scale S: NewW =
Box.width × S NewH = Box.height × S
NewX = Box.x + (Box.width - NewW) / 2
NewY = Box.y + (Box.height - NewH) / 2
Score each scaled box using color
histogram: score(S) =
compareHist(ObjHistogram,
ScaledBoxHistogram) Best scale = argmax
score(S) → Object distance
automatically handled ✓
```

10. Complete System Flow



11. Complete Feature Summary

Feature	Algorithm	Purpose
Normal speed video	Thread sleep = 1000/FPS - capture_time	Correct playback rate
Mouse ROI	LBUTTONDOWN + MOUSEMOVE + LBUTTONUP	Draw box while playing

Auto ROI	GrabCut → Canny → HSV similarity	Find object boundary
Primary tracking	CSRT (0.643 VOT accuracy)	Accurate tracking
Backup tracking	KCF (172 FPS)	Fast recovery
Occlusion predict	Kalman 6-state filter	Predict while hidden
Face recovery	Haar cascade XML	Find face after loss
Motion recovery	MOG2 background subtraction	Find moving object
Template recovery	TM_CCOEFF_NORMED ≥ 0.55	Match appearance
Object ID	Color ≥ 0.45 AND size 0.5x-2.0x	Reject wrong objects
Scale adaptation	5 scales × color score = best scale	Handle distance change

🎉 Professional-grade C++ Object Tracker
— Ready for Drone Deployment!